AI-Driven Optimization: From Algorithms to Applications

Xianchao Xiu

Department of Automation



Peking University, October 30, 2026

Joint work with Jingjing Liu (SHU), Hui Chen (SUEP), Wanquan Liu (SYSU) and others

Outline

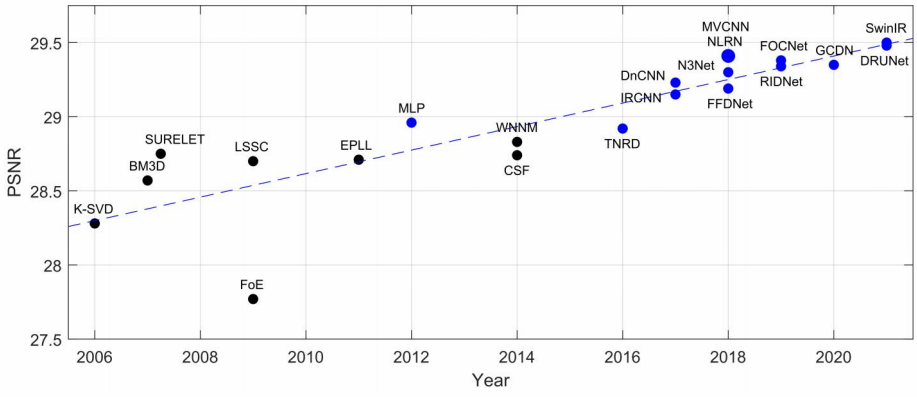
Introduction

Recent Advances

Future Work

Why

- Elad-Kawar-Vaksman, Image Denoising: The Deep Learning Revolution and Beyond, SIIMS, 2023



How

- Special Issue: Physics-Driven Machine Learning for Computational Imaging
  - Zha-Wen-Yuan et al, Learning Nonlocal Sparse and Low-Rank Models for Image Compressive Sensing: Nonlocal Sparse and Low-Rank Modeling, IEEE SPM, 2023
  - Dong-Valzania-Maillard et al, Phase Retrieval: From Computational Imaging to Machine Learning, IEEE SPM, 2023
  - Zhang-Chen-Xiong et al, Physics-Inspired Compressive Sensing: Beyond Deep Unrolling, IEEE SPM, 2023
  - Dong-Wu-Li et al, Bayesian Deep Learning for Image Reconstruction: From Structured Sparsity to Uncertainty Estimation, IEEE SPM, 2023
  - Kamilov-Bouman-Buzzard et al, Plug-and-Play Methods for Integrating Physical and Learned Models in Computational Imaging: Theory, Algorithms, and Applications, IEEE SPM, 2023
  - Chen-Davies-Ehrhardt et al, Imaging With Equivariant Deep Learning: From Unrolled Network Design to Fully Unsupervised Learning, IEEE SPM, 2023
  - Zhao-Ye-Bresler, Generative Models for Inverse Imaging Problems: From Mathematical Foundations to Physics-Driven Applications, IEEE SPM, 2023
  - Mukherjee-Hauptmann-Öktem et al, Learned Reconstruction Methods With Convergence Guarantees, IEEE SPM, 2023
- `https://github.com/xianchaoxiu/Deep-Learning-for-Optimization`

How

- Iterative shrinkage thresholding algorithm (ISTA)

$$\min_{x\in\mathbb{R}^n} \quad \frac{1}{2}\|Ax-b\|^2+\lambda\|x\|_1$$

  - Step 1: Compute gradient

$$r^{k+1}=x^k-\rho A^\top(Ax^k-b)$$

  - Step 2: Compute shrinkage

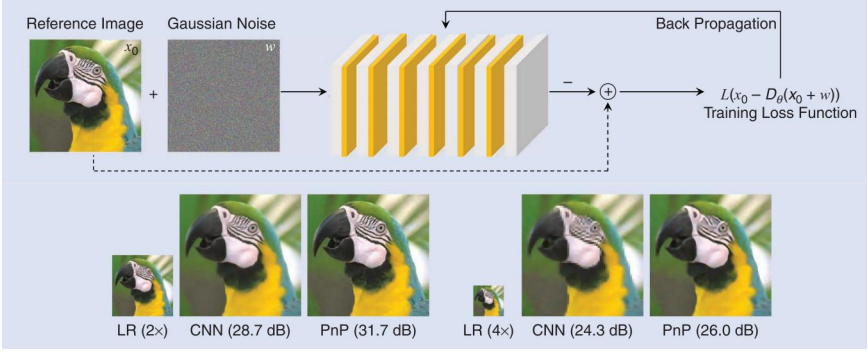
$$\begin{aligned} x^{k+1} &= \arg\min_x \ \frac{1}{2}\|x-r^{k+1}\|^2+\lambda\|x\|_1 \\ &= \mathtt{prox}_{\lambda\|\cdot\|_1}(x^k-\rho A^\top(Ax^k-b)) \end{aligned}$$

- Two important issues need to be considered
  - How to design regularizations $\Rightarrow$ Deep PnP/Deep Unrolling
  - How to choose parameters $\Rightarrow$ Deep Unrolling

Deep PnP

- Venkatakrishnan-Bouman-Wohlberg, Plug-and-Play Priors for Model Based Reconstruction, IEEE GlobalSIP, 2013

$$\min_x \quad \frac{1}{2}\|Ax-b\|^2+\lambda\|x\|_1 \quad \Rightarrow \quad \min_x \quad \frac{1}{2}\|Ax-b\|^2+\lambda\phi(x)$$



Related Works

- Ryu-Liu-Wang et al, Plug-and-Play Methods Provably Converge with Properly Trained Denoisers, ICML, 2019
- Wei-Rivero-Liang et al, Tuning-free Plug-and-Play Proximal Algorithm for Inverse Imaging Problems, ICML, 2020
- Sun-Wu-Xu et al, Scalable Plug-and-Play ADMM With Convergence Guarantees, IEEE TCI, 2021
- Cohen-Elad-Milanfar, Regularization by Denoising via Fixed-Point Projection (RED-PRO), SIIMS, 2021
- Huraul-Leclaire-Papadakis, Proximal Denoiser for Convergent Plug-and-Play Optimization with Nonconvex Regularization, ICML, 2022
- Tan-Mukherjee-Tang et al, Provably Convergent Plug-and-Play Quasi-Newton Methods, SIIMS, 2024
- Wang-Zhang-Li et al, DMPlug: A Plug-in Method for Solving Inverse Problems with Diffusion Models, NeurIPS, 2024
- Zheng-Chu-Zhang et al,InverseBench: Benchmarking Plug-and-Play Diffusion Priors for Inverse Problems in Physical Sciences, ICLR, 2025

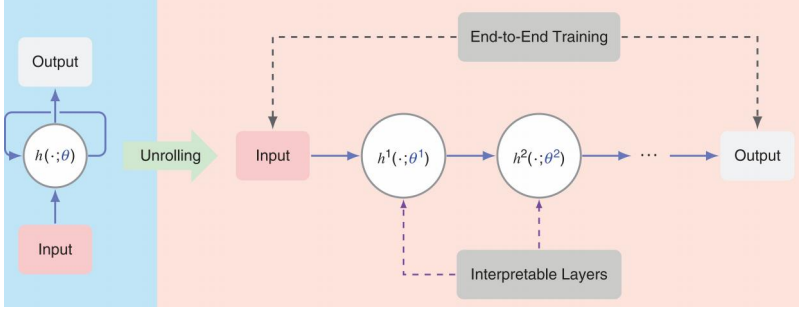
Deep Unrolling

- Gregor-LeCun, Learning Fast Approximations of Sparse Coding, ICML, 2010

$$x^{k+1} = \texttt{prox}_{\lambda\|\cdot\|_1}(x^k - \rho A^\top (Ax^k - b))$$

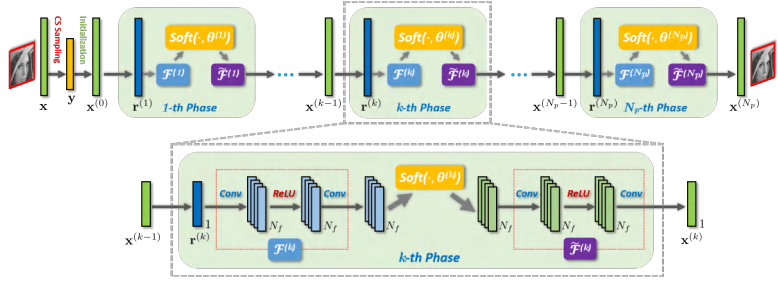
$$\Downarrow$$

$$x^{k+1} = h_{\theta^k}(x^k - \rho A^\top (Ax^k - b))$$



Deep Unrolling

- Zhang-Ghanem, ISTA-Net: Interpretable Optimization-Inspired Deep Network for Image Compressive Sensing, CVPR, 2018

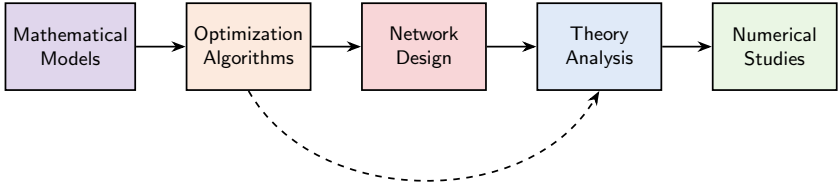


Algorithm	CS Ratio							Time
	50%	40%	30%	25%	10%	4%	1%	CPU/GPU
TVAL3 [22]	33.55	31.46	29.23	27.92	22.99	18.75	16.43	3.135s/———
D-AMP [28]	35.92	33.56	30.39	28.46	22.64	18.40	5.21	51.21s/———
IRCNN [49]	36.23	34.06	31.18	30.07	24.02	17.56	7.70	———/68.42s
SDA [30]	28.95	27.79	26.63	25.34	22.65	20.12	17.29	———/0.0032s
ReconNet [21]	31.50	30.58	28.74	25.60	24.28	20.63	17.27	———/0.016s
ISTA-Net	37.43	35.36	32.91	31.53	25.80	21.23	17.30	0.923s/0.039s
ISTA-Net$^+$	**38.07**	**36.06**	**33.82**	**32.57**	**26.64**	**21.31**	**17.34**	1.375s/0.047s

Algorithm	CS Ratio				Time
	20%	30%	40%	50%	GPU
ADMM-Net	37.17	39.84	41.56	43.00	0.9535s
ISTA-Net	38.30	40.52	42.12	43.60	0.1246s
ISTA-Net$^+$	**38.73**	**40.89**	**42.52**	**44.09**	0.1437s

Related Works

- Chen-Liu-Wang et al, Theoretical Linear Convergence of Unfolded ISTA and Its Practical Weights and Thresholds, NeurIPS, 2018
- Yang-Sun-Li et al, ADMM-CSNet: A Deep Learning Approach for Image Compressive Sensing, IEEE TPAMI, 2020
- Song-Zhang-Zhou et al, SOR-Unrolled Learning-to-Solve Method for Absolute Value Equations, JORSC, 2026
- Oshin-Ghosh-Saravanos et al, Deep FlexQP: Accelerated Nonlinear Programming via Deep Unfolding, ICLR, 2026
- Xiu-He-Qu et al, A Survey of Deep Unfolding Methods for Image Compressive Sensing (in Chinese), ORT, 2026



Outline

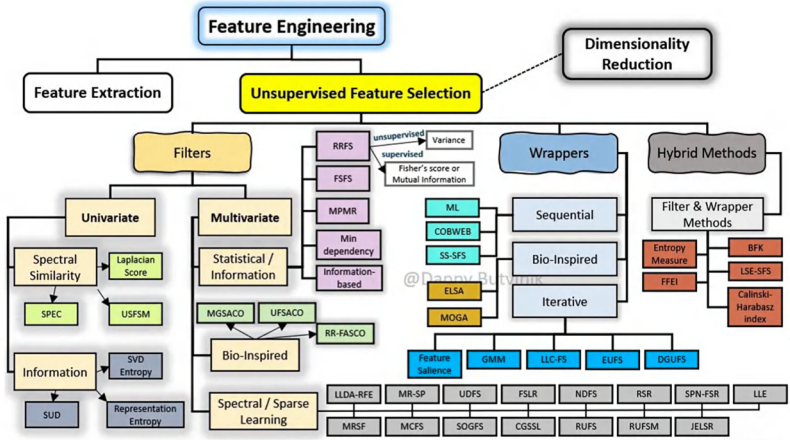
Introduction

Recent Advances

Future Work

Part I

- Unsupervised feature selection *vs.* Feature extraction
- Select a subset of input features without labels



PCA

- Given $X = (\mathbf{x}_1, \mathbf{x}_2, \ldots, \mathbf{x}_n) \in \mathbb{R}^{d \times n}$, principal component analysis (PCA) is

$$\begin{aligned} \min_{W \in \mathbb{R}^{d \times p}} \quad & -\operatorname{Tr}(W^\top X X^\top W) \\ \text{s.t.} \quad & W^\top W = I \end{aligned}$$

- Li-Nie-Bian et al, Sparse PCA via $\ell_{2,p}$-Norm Regularization for Unsupervised Feature Selection, IEEE TPAMI, 2023

$$\begin{aligned} \min_{W} \quad & -\operatorname{Tr}(W^\top X X^\top W) + \lambda \|W\|_{2,p}^p \ (0 < p < 1) \\ \text{s.t.} \quad & W^\top W = I \end{aligned}$$

- Xiu-Huang-Shang et al, Bi-Sparse Unsupervised Feature Selection, IEEE TIP, 2025

$$\begin{aligned} \min_{W} \quad & -\operatorname{Tr}(W^\top S W) + \lambda_1 \|W\|_{2,p}^p + \lambda_2 \|W\|_q^q \ (0 \le p, q < 1) \\ \text{s.t.} \quad & W^\top W = I \end{aligned}$$

Model

- Liu-Tai-Xiu et al, Learning to Select Features with Manifold Deep Unfolding PCA, submitted to IEEE TIP

$$\begin{aligned} \min_{X} \quad & \frac{1}{2}\|A - XX^\top A\|_{\mathrm{F}}^2 + \lambda\|X\|_{2,0} + \mu\|X\|_0 \\ \text{s.t.} \quad & X^\top X = I \end{aligned}$$

- Alternating direction method of multipliers (ADMM)

$$\begin{aligned} \min_{X,Y,Z} \quad & f(X) + \lambda\|Y\|_{2,0} + \mu\|Z\|_0 \\ \text{s.t.} \quad & X^\top X = I,\ X = Y,\ X = Z \end{aligned}$$

$$\Downarrow$$

$$\begin{aligned} \mathcal{L}(X, Y, Z, \Lambda, \Pi) = & f(X) + \lambda\|Y\|_{2,0} + \mu\|Z\|_0 \\ & + \langle \Lambda, X - Y\rangle + \frac{\alpha}{2}\|X - Y\|_{\mathrm{F}}^2 + \langle \Pi, X - Z\rangle + \frac{\beta}{2}\|X - Z\|_{\mathrm{F}}^2 \end{aligned}$$

Algorithm

- Update X^{k+1}

$$\begin{aligned} \min_{X} \quad & F_k(X) = f(X) + \frac{\alpha_k}{2}\|X - Y^k + \Lambda^k/\alpha_k\|_{\mathrm{F}}^2 + \frac{\beta_k}{2}\|X - Z^k + \Pi^k/\beta_k\|_{\mathrm{F}}^2 \\ \text{s.t.} \quad & X^\top X = I \end{aligned}$$

Let $G = AA^\top$. Then

$$f(X) = \frac{1}{2}\|A\|_{\mathrm{F}}^2 - \frac{1}{2}\mathrm{Tr}(X^\top G X)$$

The Euclidean gradient is

$$\nabla F_k(X) = -GX + \alpha_k(X - Y^k + \Lambda^k/\alpha_k) + \beta_k(X - Z^k + \Pi^k/\beta_k)$$

The projected search direction is

$$D_k = \mathcal{P}_{X^k}(\nabla F_k(X^k)) = (I - X^k(X^k)^\top)\nabla F_k(X^k)$$

Algorithm

- **Input:** Data A, stage-wise parameters $\lambda_k, \mu_k, \alpha_k, \beta_k, \eta_k$
- **Initialize:** $X^0, Y^0, Z^0, \Lambda^0, \Pi^0$
  - Update X^{k+1} according to

$$X^{k+1} = \texttt{MGradNet+}(X^k, Y^k, Z^k, \Lambda^k, \Pi^k, G, \alpha_k, \beta_k, \eta_k)$$

  - Update Y^{k+1} according to

$$Y^{k+1} = \texttt{HardNet}(U^k, \tau_k)$$

  - Update Z^{k+1} according to

$$Z^{k+1} = \texttt{EHardNet}(V^k, \gamma_k)$$

  - Update Λ^{k+1} according to

$$\Lambda^{k+1} = \texttt{Linear}(\Lambda^k, X^{k+1}, Y^{k+1}, \alpha_k)$$

  - Update Π^{k+1} according to

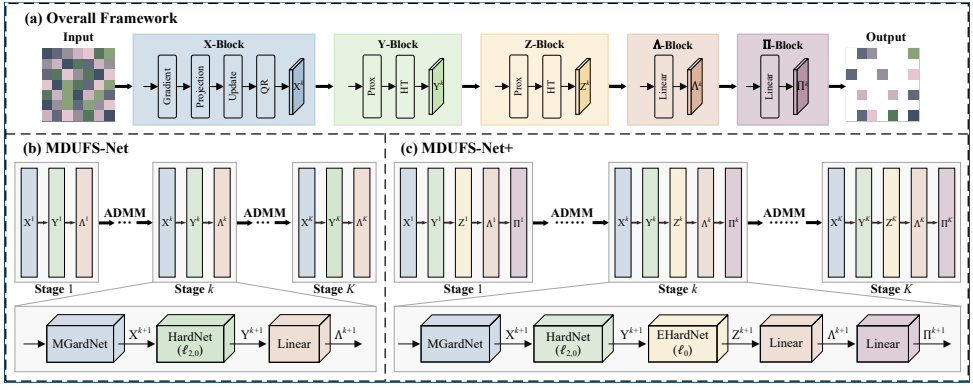
$$\Pi^{k+1} = \texttt{Linear}(\Pi^k, X^{k+1}, Z^{k+1}, \beta_k)$$

- **Output:** X^K, Y^K, Z^K

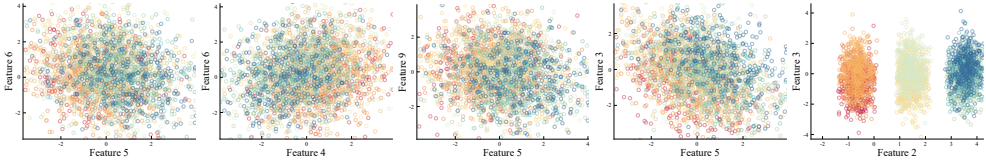
Architecture

- All papameters $(\lambda, \mu, \alpha, \beta)$ are trained in an end-to-end manner
- The loss is defined as

$$\text{Loss} = f(X) + \lambda\varphi_{2,0}(Y) + \mu\varphi_0(Z)$$



Demo



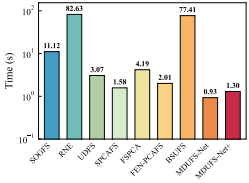
(a) LapScore (b) SOGFS (c) RNE (d) UDFS (e) SPCAFS

(f) FSPCA (g) FEN-PCAFS (h) BSUFS (i) MDUFS-Net (j) MDUFS-Net+

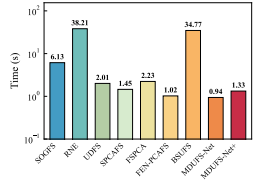
Results

Methods	COIL20	ISOLET	USPS	PIE	LUNG	MSTAR	MNIST	HAR	Average
ALLfea	64.73±7.63 (10)	59.87±2.78 (10)	59.82±1.29 (10)	27.43±0.96 (10)	72.57±7.63 (10)	90.29±3.94 (10)	47.69±2.78 (10)	53.79±0.99 (10)	59.52±3.50
LapScore	55.36±1.58 (90)	54.07±1.80 (100)	52.31±2.68 (100)	33.90±0.97 (30)	58.40±0.75 (100)	68.67±3.32 (100)	48.62±0.15 (80)	61.59±0.09 (100)	54.12±1.42
SOGFS	43.32±2.96 (100)	43.43±1.84 (30)	45.23±1.31 (100)	24.73±0.94 (100)	54.02±3.19 (100)	64.65±2.66 (60)	41.26±1.15 (80)	46.05±0.38 (100)	45.34±1.80
RNE	56.64±3.40 (80)	50.29±1.77 (100)	57.51±2.01 (100)	22.39±0.61 (60)	58.90±5.79 (100)	69.11±1.02 (80)	39.58±0.19 (100)	42.16±0.03 (60)	49.57±1.85
UDFS	48.38±2.41 (100)	40.84±1.83 (100)	50.63±1.93 (100)	27.49±0.94 (100)	58.19±5.29 (100)	75.24±3.37 (90)	34.57±1.37 (80)	42.03±0.03 (100)	47.17±2.15
SPCAFS	45.72±2.07 (80)	40.55±1.76 (100)	57.95±1.57 (80)	45.41±1.74 (80)	56.32±4.29 (90)	67.96±2.81 (100)	31.71±0.91 (90)	58.67±0.01 (90)	50.54±1.90
FSPCA	56.57±1.89 (80)	47.99±1.31 (50)	56.05±1.96 (60)	32.45±1.18 (80)	55.74±1.90 (90)	80.09±5.55 (70)	49.49±1.43 (70)	44.54±0.05 (40)	52.87±1.91
FEN-PCAFS	58.20±2.24 (100)	52.28±2.11 (100)	58.28±2.31 (100)	38.94±1.64 (80)	61.77±1.53 (80)	81.07±3.84 (80)	47.66±0.02 (60)	60.91±0.04 (20)	57.39±1.72
BSUFS	57.26±2.37 (100)	53.59±2.12 (100)	59.33±0.96 (80)	45.25±1.74 (80)	60.13±2.42 (70)	71.45±1.81 (100)	33.80±0.89 (100)	63.27±0.05 (90)	55.51±1.55
MDUFS-Net	**65.60±2.83 (100)**	62.84±2.27 (100)	62.02±1.69 (100)	45.61±1.73 (70)	66.12±1.86 (100)	88.00±1.83 (90)	56.10±0.05 (100)	67.53±0.25 (40)	64.23±1.56
MDUFS-Net+	65.23±1.89 (90)	**64.16±2.04 (100)**	**64.79±1.65 (80)**	**46.82±1.65 (60)**	**69.46±1.35 (60)**	**91.70±0.15 (100)**	**56.75±0.03 (100)**	**70.19±0.13 (20)**	**66.14±1.11**

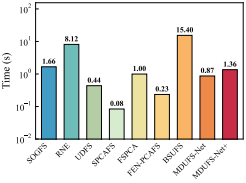
Runtime



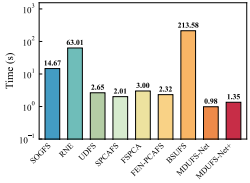
(a) COIL20



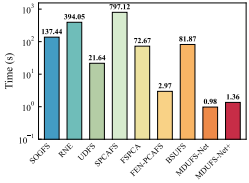
(b) ISOLET



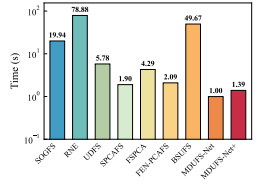
(c) USPS



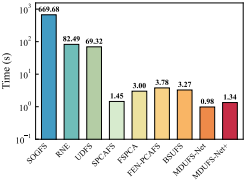
(d) PIE



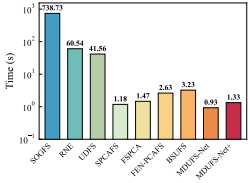
(e) LUNG



(f) MSTAR



(g) MNIST



(h) HAR

Part II

- Wu-Zhang-Li et al, RPCANet: Deep Unfolding RPCA Based Infrared Small Target Detection, WACV, 2024

$$\begin{aligned} \min_{B,T} \quad & \|B\|_* + \lambda \|T\|_1 \\ \text{s.t.} \quad & D = B + T \end{aligned}$$



Model

- Liu-Han-Xiu et al, Lightweight Deep Unfolding Networks with Enhanced Robustness for Infrared Small Target Detection, IEEE TIP, AQ

$$\begin{aligned} \min_{B,T} \quad & \|B\|_* + \lambda \|T\|_1 \\ \text{s.t.} \quad & D = B + T \end{aligned}$$

$$\Downarrow$$

$$\begin{aligned} \min_{B,T,N} \quad & \|B\|_* + \lambda \|T\|_1 + \mu \|N\|_F^2 \\ \text{s.t.} \quad & D = B + T + N \end{aligned}$$

$$\Downarrow$$

$$\begin{aligned} \min_{B,T,N} \quad & \mathcal{R}(B) + \lambda \mathcal{S}(T) + \mu \mathcal{G}(N) \\ \text{s.t.} \quad & D = B + T + N \end{aligned}$$

- Unconstrained version

$$\mathcal{L}(B, T, N) = \mathcal{R}(B) + \lambda \mathcal{S}(T) + \mu \mathcal{G}(N) + \frac{\alpha}{2} \|D - B - T - N\|_{\mathrm{F}}^2$$

Update B

- `SEBEM`: Bcakgound estimation module (BEM) + squeeze-and-excitation network (SE)

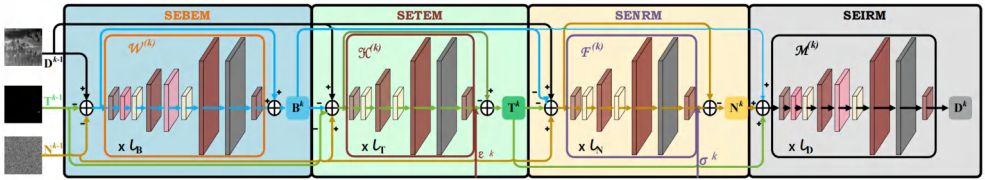
$$B^k = \arg\min_B \ \mathcal{R}(B) + \frac{\alpha}{2}\|D^{k-1} - B - T^{k-1} - N^{k-1}\|_{\mathrm{F}}^2$$

$$\Downarrow$$

$$B^k = \mathrm{prox}_{\alpha}(D^{k-1} - T^{k-1} - N^{k-1})$$

$$\Downarrow$$

$$B^k = D^{k-1} - T^{k-1} - N^{k-1} + \mathcal{W}^k(D^{k-1} - T^{k-1} - N^{k-1})$$



Update T

- SETEM: Target estimation module (TEM) + squeeze-and-excitation network (SE)

$$T^k = \arg\min_T \ \lambda \mathcal{S}(T) + \frac{\alpha}{2}\|D^{k-1} - B^k - T - N^{k-1}\|_{\mathrm{F}}^2$$

$$\Downarrow$$

$$T^k = \arg\min_T \ \frac{\lambda L_{\mathcal{S}}}{2}\|T - T^{k-1} + \frac{1}{L_{\mathcal{S}}}\nabla\mathcal{S}(T^{k-1})\|_{\mathrm{F}}^2 + \frac{\alpha}{2}\|D^{k-1} - B^k - T - N^{k-1}\|_{\mathrm{F}}^2$$

$$\Downarrow$$

$$T^k = \frac{\lambda L_{\mathcal{S}}}{\lambda L_{\mathcal{S}} + \alpha} T^{k-1} + \frac{\alpha}{\lambda L_{\mathcal{S}} + \alpha}\left(D^{k-1} - B^k - N^{k-1}\right) - \frac{\lambda}{\lambda L_{\mathcal{S}} + \alpha}\nabla\mathcal{S}(T^{k-1})$$

$$\Downarrow$$

$$T^k = \gamma T^{k-1} + (1-\gamma)(D^{k-1} - B^k - N^{k-1}) - \varepsilon\nabla\mathcal{S}(T^{k-1})$$

Update T

- Set $\gamma = 0.5$

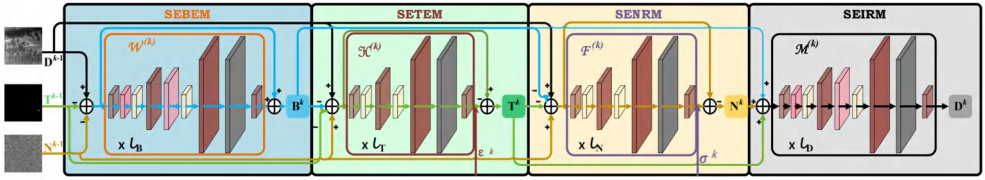
$$T^k = \gamma T^{k-1} + (1-\gamma)(D^{k-1} - B^k - N^{k-1}) - \varepsilon \nabla \mathcal{S}(T^{k-1})$$

$$\Downarrow$$

$$T^k = T^{k-1} + D^{k-1} - B^k - N^{k-1} - \varepsilon \nabla \mathcal{S}(T^{k-1})$$

$$\Downarrow$$

$$T^k = T^{k-1} + D^{k-1} - B^k - N^{k-1} - \varepsilon^k \mathcal{H}^k(T^{k-1} + D^{k-1} - B^k - N^{k-1})$$



Update N

- `SENRM`: Noise reduction module (NRM) + squeeze-and-excitation network (SE)

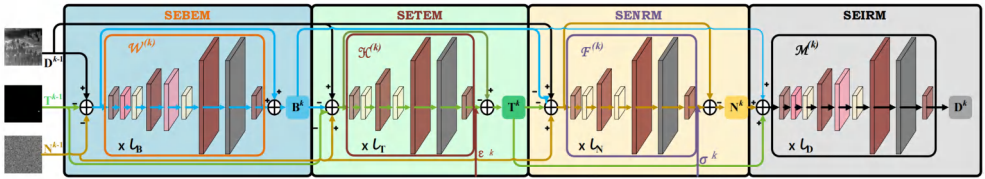
$$N^k = \arg\min_N \ \mu\mathcal{G}(N) + \frac{\alpha}{2}\|D^{k-1} - B^k - T^k - N\|_{\mathrm{F}}^2$$

$$\Downarrow$$

$$N^k = \arg\min_N \ \frac{\mu L_N}{2}\|N - N^{k-1} + \frac{1}{L_N}\nabla\mathcal{G}(N^{k-1})\|_{\mathrm{F}}^2 + \frac{\alpha}{2}\|D^{k-1} - B^k - T^k - N\|_{\mathrm{F}}^2$$

$$\Downarrow$$

$$N^k = N^{k-1} + D^{k-1} - B^k - T^k - \sigma^k\mathcal{F}^k(N^{k-1} + D^{k-1} - B^k - T^k)$$



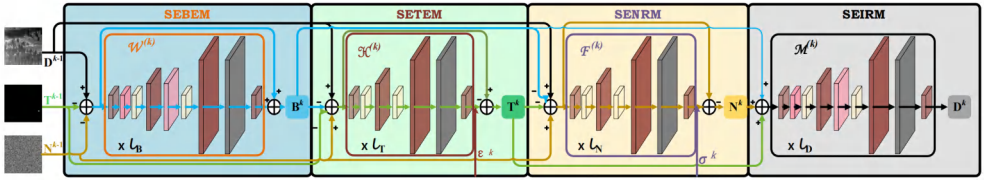
Update D

- `SEIRM`: Image reconstruction module (IRM)+ squeeze-and-excitation network (SE)

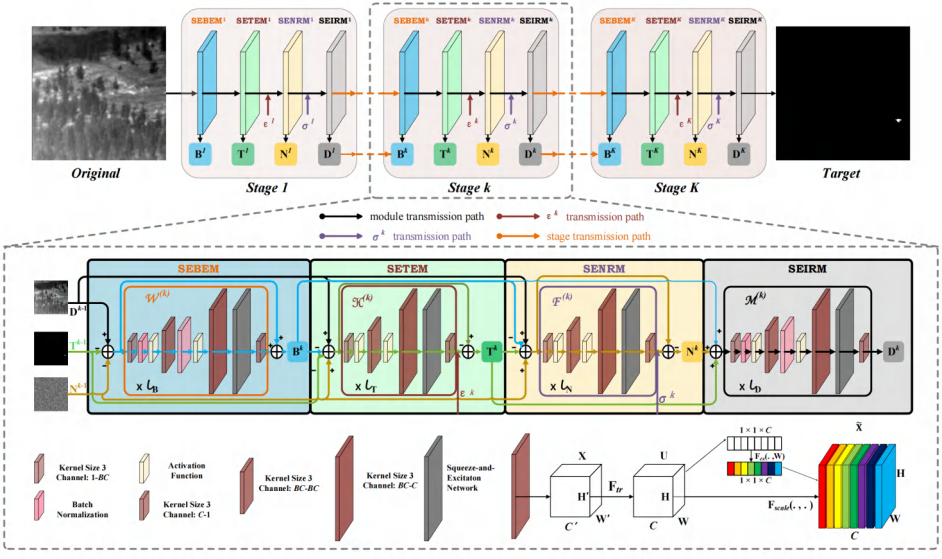
$$D^k = B^k + T^k + N^k$$

$$\Downarrow$$

$$D^k = \mathcal{M}^k(B^k + T^k + N^k)$$

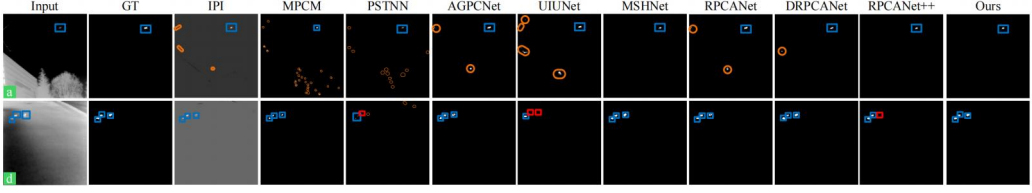


Architecture



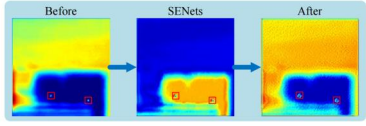
Results

Methods	Params	NUDT-SIRST				SIRST-Aug				IRSTD-1k				Time (s)
		mIoU ↑	F_1 ↑	P_d ↑	F_a ↓	mIoU ↑	F_1 ↑	P_d ↑	F_a ↓	mIoU ↑	F_1 ↑	P_d ↑	F_a ↓	CPU/GPU
IPI	–	34.83	51.49	92.58	7.14	21.90	35.97	80.36	**2.20**	18.67	31.48	78.54	11.11	3.0972/-
MPCM	–	25.96	40.78	78.59	7.91	19.49	33.00	93.58	3.04	14.81	25.93	69.03	6.51	0.0624/-
PSTNN	–	25.46	40.58	78.52	7.95	19.76	33.00	93.40	3.14	14.87	25.89	68.73	6.51	0.2249/-
AGPCNet	12.360M	85.31	92.45	97.90	4.77	72.36	83.83	99.03	35.56	61.00	75.75	89.35	5.34	-/0.0205
UIUNet	50.540M	88.71	94.01	91.43	1.89	71.80	83.59	98.35	28.29	63.06	77.35	**93.60**	6.57	-/0.0317
MSHNet	4.065M	89.99	93.57	96.07	2.63	71.64	84.16	90.78	23.09	64.50	77.55	91.68	4.46	-/0.0245
RPCANet	0.680M	89.31	94.35	97.14	2.87	72.54	84.08	98.21	34.14	63.21	77.45	88.31	4.39	-/0.0096
DRPCANet	1.169M	**93.12**	96.02	98.02	1.95	73.93	85.39	98.12	30.45	63.93	78.15	92.09	4.92	-/0.0101
RPCANet++	4.396M	92.46	96.05	98.05	**1.44**	73.14	84.39	97.36	32.48	64.03	77.26	89.35	**4.28**	-/0.0063
Ours	**0.216M**	92.37	**96.54**	**98.41**	1.79	**74.56**	**85.43**	**99.17**	29.78	**64.68**	**78.55**	89.39	4.66	**-/0.0052**

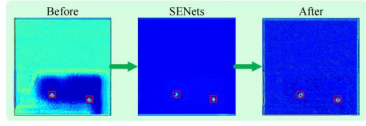


Ablations

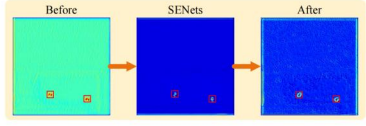
SENets				NUDT-SIRST				SIRST-Aug				IRSTD-1k			
SEBEM	SETEM	SENRM	SEIRM	mIoU ↑	F_1 ↑	P_d ↑	F_a ↓	mIoU ↑	F_1 ↑	P_d ↑	F_a ↓	mIoU ↑	F_1 ↑	P_d ↑	F_a ↓
✗	✗	✗	✗	73.56	78.45	79.36	8.56	60.75	70.28	81.24	43.67	50.26	61.34	70.57	15.95
✓	✗	✗	✗	80.57	81.39	86.35	5.68	65.96	75.37	86.99	39.82	55.84	65.26	76.36	11.12
✓	✓	✗	✗	88.36	89.45	90.18	4.00	70.17	80.78	93.17	34.78	60.56	72.58	83.70	8.10
✓	✓	✓	✗	91.14	96.06	97.18	2.05	73.27	84.45	98.07	27.73	63.58	77.53	88.71	3.95
✓	✓	✓	✓	**92.37**	**96.54**	**98.41**	**1.79**	**74.56**	**85.43**	**99.17**	**29.78**	**64.68**	**78.55**	**89.39**	**4.66**



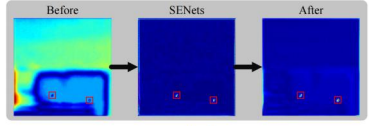
(a) SEBEM



(b) SETEM



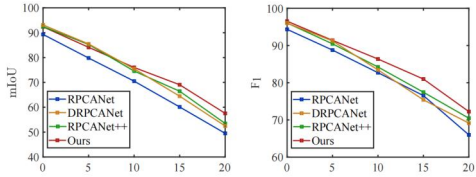
(c) SENRM



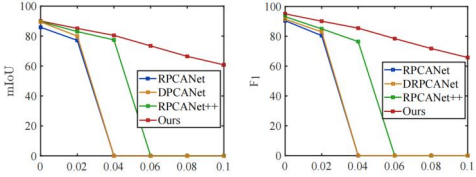
(d) SEIRM

Robustness

- Gaussian noise



- Salt-and-pepper noise

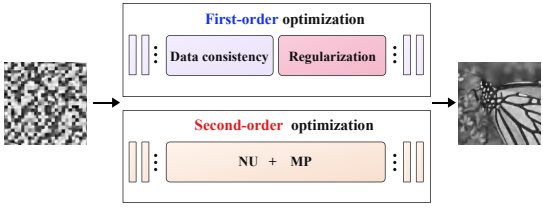


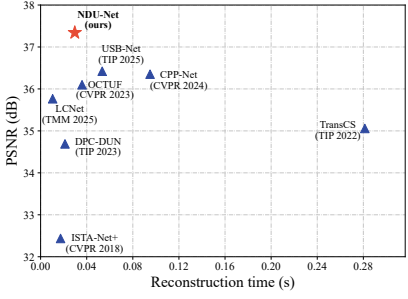
Part III

- He-Xiu, Newton Deep Unfolding for Compressed Sensing, submitted to AAAI

$$\min_{x\in\mathbb{R}^n} \underbrace{\frac{1}{2}\|Ax-b\|^2}_{f(x)} + \lambda\mathcal{R}(x)$$

- Iterative optimization: ISTA, FISTA, AMP, PDHG, ADMM, SSNAL, NHTP
- Deep unrolling: LISTA, ALISTA, ISTA-Net, ISTA-Net+, LDAMP, ADMM-CSNet





Step 1

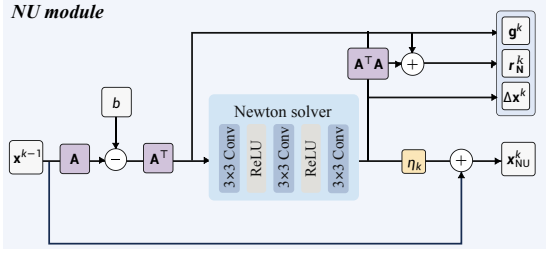
- *(ISTA)* Compute gradient

$$r^{k+1} = x^k - \rho \nabla f(x^k)$$

- *(NDU-Net)* The Newton update (NU) satisfies

$$\begin{cases} \nabla^2 f(x^k)\Delta x^k + \nabla f(x^k) = 0 \\ x^k_{\mathrm{NU}} \leftarrow x^k + \eta_k \Delta x^k \end{cases}$$

Define the Newton residual as $r_N^k = A^\top A \Delta x^k + \nabla f(x^k)$



Step 2

- *(ISTA)* Compute soft-thresholding

$$x^{k+1} = \texttt{Prox}_{\lambda\|\cdot\|_1}(x^k - \rho\nabla f(x^k))$$

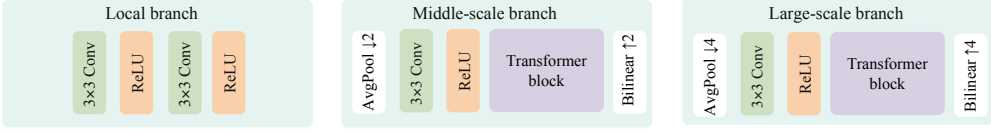
- *(NDU-Net)* Extract multi-scale prior (MP) information

$$y_{MS}^k = \mathcal{P}_{\mathrm{MS}}^k(x_{\mathrm{NU}}^k)$$

Encode Newton guidance information

$$y_N^k = \mathcal{E}_N^k(g^k, r_N^k, \Delta x^k)$$

$$\Rightarrow \quad y_{\mathrm{guided}}^k = y_{MS}^k + y_N^k$$



Step 2

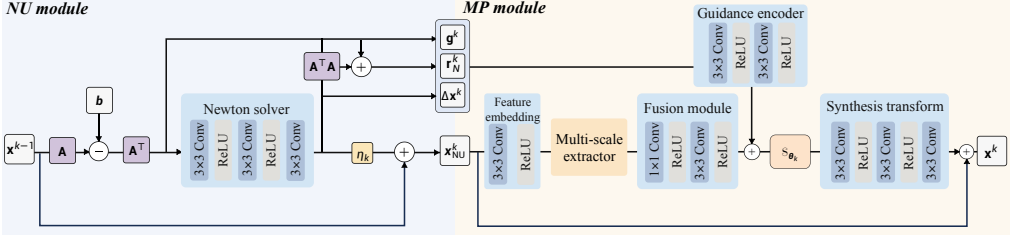
- *(NDU-Net)* Apply a learnable soft-thresholding operation

$$y^k_{\text{shrink}} = \mathcal{S}_{\theta_k}(y^k_{\text{guided}})$$

$$\mathcal{S}_{\theta_k}(z) = \text{sign}(z) \odot \max(|z| - \theta_k, 0)$$

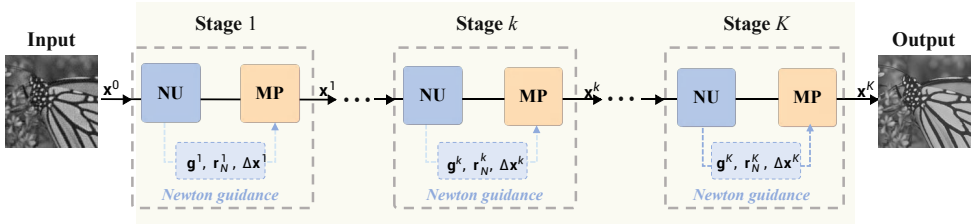
Transform back to the image domain

$$r^k_{\text{prior}} = \mathcal{H}^k(y^k_{\text{shrink}})$$



Architecture

- Overall architecture



- In the k-th stage

$$x^{k+1} = x^k_{\text{NU}} \text{ (Step 1)} + r^k_{\text{prior}} \text{ (Step 2)}$$

Training loss

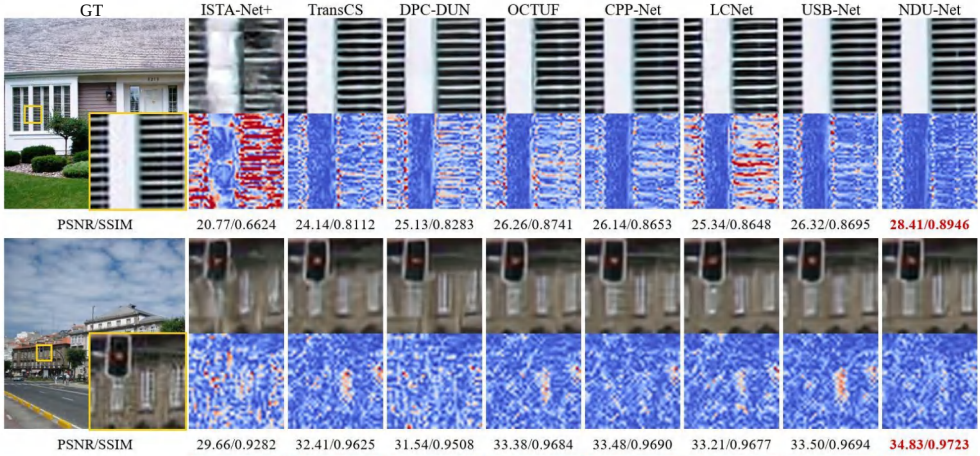
$$\ell_{\text{total}} = \ell_{\text{rec}} + \sigma \ell_{\text{Newton}}$$

$$= \frac{1}{N}\|x^K - x\|_1 + \sigma \left(\frac{1}{K} \sum_{k=0}^{K-1} \left(\frac{1}{N} \|r_N^k\|_1 \right) \right)$$

Results

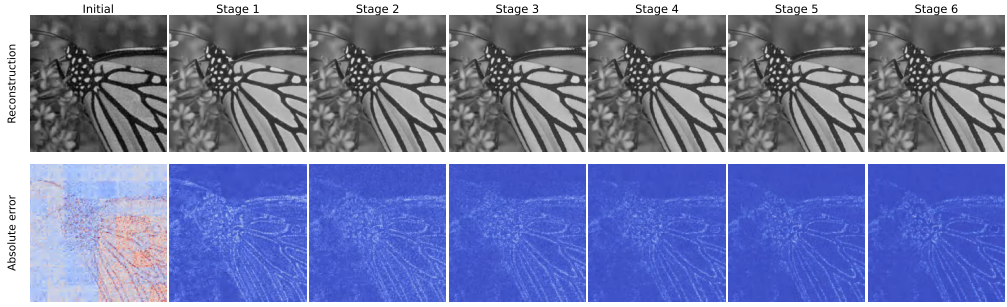
Datsets	Methods	CS ratios 0.10	0.25	0.40	0.50	Average
LIVE29	ISTA-Net+ (CVPR'18)	25.05/0.7104	29.42/0.8648	32.55/0.9275	34.55/0.9514	30.39/0.8635
	TransCS (TIP'22)	27.66/0.8196	32.05/0.9251	35.51/0.9633	37.74/0.9767	33.24/0.9212
	DPC-DUN (TIP'23)	26.83/0.7812	31.17/0.9008	34.43/0.9477	36.43/0.9649	32.22/0.8987
	OCTUF (CVPR'23)	28.23/0.8345	32.69/0.9327	36.19/0.9671	38.52/0.9795	33.91/0.9285
	CPP-Net (CVPR'24)	28.46/0.8414	32.82/0.9343	36.20/0.9672	38.37/0.9790	33.96/0.9305
	LCNet (TMM'25)	28.19/0.8349	32.64/0.9325	36.08/0.9667	38.34/0.9791	33.81/0.9283
	USB-Net (TIP'25)	28.48/0.8420	32.87/0.9348	36.29/0.9676	38.46/0.9793	34.03/0.9309
	NDU-Net	**29.62/0.8517**	**34.07/0.9399**	**37.55/0.9708**	**39.62/0.9812**	**35.22/0.9359**
OST300	ISTA-Net+ (CVPR'18)	24.78/0.6896	28.53/0.8433	31.34/0.9116	33.20/0.9396	29.46/0.8460
	TransCS (TIP'22)	27.31/0.8019	31.07/0.9096	34.17/0.9534	36.25/0.9701	32.20/0.9088
	DPC-DUN (TIP'23)	26.25/0.7561	29.93/0.8792	32.81/0.9335	34.69/0.9554	30.92/0.8811
	OCTUF (CVPR'23)	27.77/0.8148	31.60/0.9175	34.61/0.9572	36.69/0.9726	32.67/0.9155
	CPP-Net (CVPR'24)	27.93/0.8207	31.67/0.9185	34.66/0.9573	36.68/0.9724	32.74/0.9172
	LCNet (TMM'25)	27.73/0.8151	31.53/0.9169	34.53/0.9568	36.60/0.9723	32.60/0.9153
	USB-Net (TIP'25)	27.95/0.8214	31.71/0.9191	34.71/0.9578	36.71/0.9726	32.77/0.9177
	NDU-Net	**29.22/0.8314**	**33.00/0.9239**	**36.01/0.9604**	**37.94/0.9739**	**34.04/0.9224**

Visualization



Remarks

- How to automatically determine the number of stages?

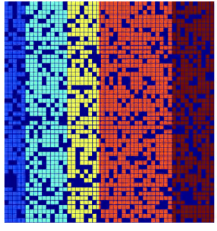
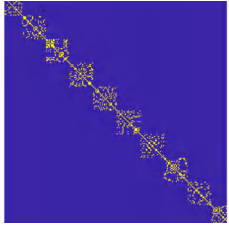
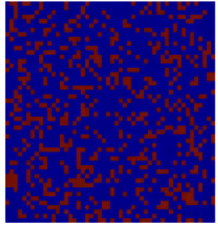


- Does it exhibit quadratic convergence under some conditions?
- What is the connection between learned solvers and true Newton directions?

Part IV

- Liu-Lin-Yan et al, Robust Recovery of Subspace Structures by Low-Rank Representation, IEEE TPAMI, 2013

$$\min_{Z,E} \ \operatorname{rank}(Z) + \lambda \|E\|_0 \quad \text{s.t.} \ \ X = XZ + E \quad \Rightarrow \quad \min_{Z,E} \ \|Z\|_* + \lambda \|E\|_1 \quad \text{s.t.} \ \ X = XZ + E$$

- Board applications: clustering, classification, denoising, detection, fusion
- Related to low-rank regularization, low-rank approximation, low-rank decomposition

Model

- Chen-Wang-Xiu et al, Rethinking Low-Rank Representation with Deep Semi-Smooth Newton, PRCV, 2026

$$\begin{aligned} \min_{Z,E} \ & \|Z\|_* + \lambda\|E\|_1 \\ \text{s.t.} \ & X = XZ + E \end{aligned} \quad \Rightarrow \quad \begin{aligned} \min_{Z,E} \ & \mathcal{R}(Z) + \lambda\mathcal{S}(E) \\ \text{s.t.} \ & X = XZ + E \end{aligned}$$

- Augmented Lagrangian method (ALM)

$$\begin{aligned} \min_{J,E,Z} \ & \mathcal{R}(J) + \lambda\mathcal{S}(E) \\ \text{s.t.} \ & X = XZ + E, \ Z = J \end{aligned}$$

which derives

$$\begin{aligned} \mathcal{L}_\beta(J, E, Z, Y_1, Y_2) &= \mathcal{R}(J) + \lambda\mathcal{S}(E) + \langle Y_1, Z - J\rangle \\ &+ \frac{\beta}{2}\|Z - J\|_F^2 + \langle Y_2, X - XZ - E\rangle + \frac{\beta}{2}\|X - XZ - E\|_F^2 \end{aligned}$$

Algorithm

- Update variables according to

$$\begin{cases} (J, E, Z) \leftarrow \arg\min_{J,E,Z} \ \mathcal{L}_\beta(J, E, Z, Y_1, Y_2) \\ Y_1 \leftarrow Y_1 + \beta(Z - J) \\ Y_2 \leftarrow Y_2 + \beta(X - XZ - E) \end{cases}$$

- First-order optimality conditions

$$\begin{cases} 0 \in \partial\mathcal{R}(J) - Y_1 + \beta(J - Z) \\ 0 \in \lambda\,\partial\mathcal{S}(E) - Y_2 - \beta(X - XZ - E) \\ 0 = Y_1 + \beta(Z - J) - X^\top Y_2 - \beta X^\top (X - XZ - E) \end{cases}$$

which derives

$$J = \mathtt{Prox}_{1/\beta,\mathcal{R}}\left(Z + Y_1/\beta\right), \quad E = \mathtt{Prox}_{\lambda/\beta,\mathcal{S}}\left(X - XZ + Y_2/\beta\right)$$

SSN

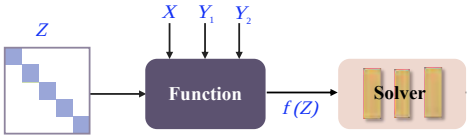
- Denote the nonlinear equation

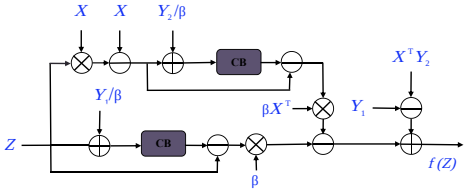
$$f(Z) = Y_1 + \beta\left(Z - \texttt{Prox}_{1/\beta,\mathcal{R}}(Z + Y_1/\beta)\right) - X^\top Y_2 \\ - \beta X^\top\left(X - XZ - \texttt{Prox}_{\lambda/\beta,\mathcal{S}}(X - XZ + Y_2/\beta)\right)$$

$$\Downarrow$$

$$\begin{cases} g(Z)\Delta Z = -f(Z) \\ Z \leftarrow Z + \eta\Delta Z \end{cases}$$

- Detailed structure: NS (Newton solver), CB (convolutional block)



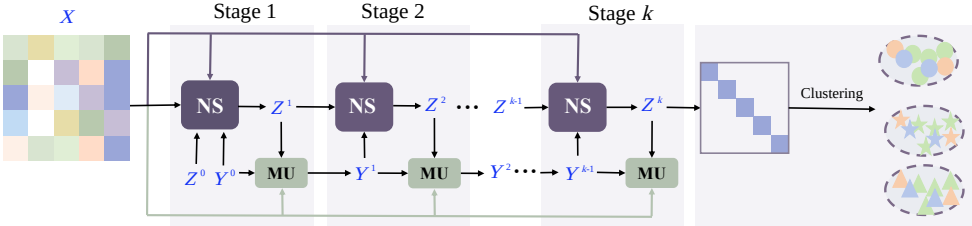


Architecture

- Training loss

$$\ell_{\text{total}} = \ell_{\text{self}} + \sigma_1 \ell_{\text{fun}} + \sigma_2 \ell_{\text{reg}}$$

$$= \left(\|X - XZ^K - E^K\|_F^2 + \gamma \|E^K\|_1 \right) + \sigma_1 \left(\frac{1}{N} \sum_{l=1}^{N} \|f(Z^K)\|_1 \right) + \sigma_2 \text{tr}(L_{Z^K})$$



Results

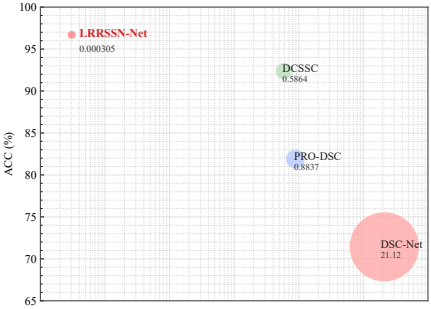
Methods	Datasets								Average	
	AR	FRDUE	MNIST	USPS	GLOMA	ISOLET	MSTAR	TDT2	ACC	Time
IDR (TKDE'23)	72.64%	95.31%	58.61%	79.80%	66.00%	67.65%	84.50%	84.10%	76.08%	225.80
TS1-LLRR (KBS'24)	76.46%	88.64%	49.65%	79.48%	61.60%	65.64%	87.61%	84.51%	74.20%	265.10
JSSC (ESWA'25)	78.09%	91.31%	46.78%	69.22%	50.00%	66.61%	65.99%	59.37%	65.92%	358.60
ISC (TPAMI'26)	54.42%	88.19%	44.02%	65.30%	60.00%	81.02%	43.13%	51.75%	60.98%	118.50
DSC-Net (NeurIPS'17)	45.19%	82.53%	66.07%	90.00%	70.00%	41.41%	82.53%	12.45%	61.27%	23.23
PRO-DSC (ICLR'25)	63.75%	**96.09%**	45.39%	67.60%	59.40%	52.00%	66.41%	88.08%	67.34%	**0.03**
DCSSC (Neuro'25)	83.68%	80.46%	87.25%	90.30%	84.00%	86.00%	91.85%	78.35%	85.24%	0.85
LRRSSN-Net	**89.95%**	96.02%	**93.43%**	**99.40%**	**98.00%**	**97.72%**	**99.72%**	**96.40%**	**96.33%**	0.21



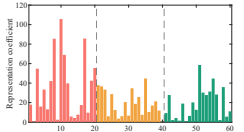
(a) IDR (b) TS1-LLRR (c) JSSC (d) ISC (e) DSC-Net (f) PRO-DSC (g) DCSSC (h) LRRSSN-Net

Analysis

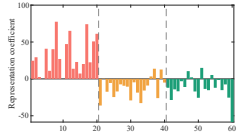
- Parameter efficiency



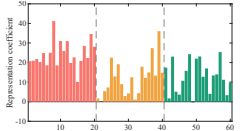
- Representation efficiency



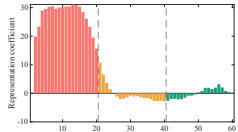
(a) TS1-LLRR



(b) JSSC



(c) DSC-Net



(d) LRRSSN-Net

Remarks

- Chen-Wang-Xiu et al, LLM-IMVC: Large Language Model-Guided Deep Unrolling for Incomplete Multi-View Clustering, working in process

$$\min_{Z,E} \ \mathcal{R}(Z) + \lambda \mathcal{S}(E) \quad \text{s.t.} \ \ X = XZ + E \qquad \Rightarrow \qquad \min_{Z_v, E_v} \ \sum_{v=1}^{V} \mathcal{R}(Z_v) + \lambda \mathcal{S}(E_v) \quad \text{s.t.} \ \ \mathcal{P}_\Omega(X_v) = \mathcal{P}_\Omega(X_v Z_v + E_v)$$

Methods	HandWrite	BDGP	MSRC	COIL20	Caltech5V	USPS	Average
LMVSC (AAAI'20)	67.1%	55.6%	35.2%	69.2%	46.1%	56.0%	54.9%
SFMC (TPAMI'22)	75.7%	37.8%	60.0%	74.8%	46.8%	98.9%	65.7%
FPMVS-CAG (TIP'22)	82.3%	55.6%	80.5%	60.2%	80.9%	78.3%	73.0%
FDAGF (AAAI'23)	82.3%	52.5%	73.9%	74.0%	81.3%	56.1%	70.0%
RCAGL (TKDE'24)	79.4%	51.8%	77.6%	65.1%	83.6%	60.9%	69.7%
ALPC (AAAI'25)	63.2%	92.2%	64.3%	76.4%	46.1%	50.2%	65.4%
TLRLF4 (TPAMI'25)	76.4%	92.7%	—	76.3%	75.5%	99.2%	84.0%
LLM-IMVC	**99.9%**	**100.0%**	**88.1%**	**77.2%**	**100.0%**	**99.9%**	**94.2%**

Part V

- LLM for Optimization & Optimization for LLM

对于提出的联邦 PCA 模型，请设计 ADMM 算法，提供代码，并分析收敛性和复杂度
\begin {equation}\label {eq:proposed_method}
\begin {array}{cl}
\displaystyle \min_{{\mathbf {W}_i}, {\mathbf {S} i}, \mathbf {V}} \quad & \sum\limits {i=1}^d \big {| (\mathbf {I} - \mathbf {W} i \mathbf {W}{i}^{T}){\mathbf {X}_i - \mathbf {S}_i} |_F^2
& + ~\alpha |\mathbf{S}_i|_1 + \beta |\mathbf{W}i{2,1}\big}{6pt}
\textrm{s.t.} & \mathbf{W}_i = \mathbf{V}, \quad \mathbf{W}i^{T} \mathbf{W}{i}=\mathbf{I}, \quad \forall i \in [d],
\end{array}
\end{equation}

联邦鲁棒 PCA：ADMM 完整设计 + Python 可运行代码 + 收敛性分析 + 计算复杂度

整体框架概述

符号约定

- d：客户端数量；第 i 方数据 $\boldsymbol{X}_i \in \mathbb{R}^{n_i \times p}$，$n_i$：每方样本数，$p$：特征维度
- $\boldsymbol{W}_i \in \mathbb{R}^{p \times k}$：客户端局部投影矩阵，满足 Stiefel 正交约束 $\boldsymbol{W}_i^\top \boldsymbol{W}_i = \boldsymbol{I}_k$
- $\boldsymbol{V} \in \mathbb{R}^{p \times k}$：全局共享公共投影矩阵（联邦耦合变量）
- $\boldsymbol{S}_i \in \mathbb{R}^{n_i \times p}$：稀疏噪声 / 异常矩阵，$\ell_1$ 正则去除粗差
- $\|\boldsymbol{W}_i\|_{2,1}$：行稀疏正则，让投影基具备行稀疏性
- $\alpha > 0$：稀疏权重；$\beta > 0$：行稀疏权重；$\rho > 0$：ADMM 增广拉格朗日惩罚系数

约束：$\forall i, \boldsymbol{W}_i = \boldsymbol{V},\ \boldsymbol{W}_i^\top \boldsymbol{W}_i = \boldsymbol{I}$

原目标：

$$\min_{\{\boldsymbol{W}_i,\boldsymbol{S}_i\},\boldsymbol{V}} \sum_{i=1}^{d} \left\{ \left\| (\boldsymbol{I} - \boldsymbol{W}_i\boldsymbol{W}_i^\top)(\boldsymbol{X}_i - \boldsymbol{S}_i) \right\|_F^2 + \alpha\|\boldsymbol{S}_i\|_1 + \beta\|\boldsymbol{W}_i\|_{2,1} \right\}, \quad \text{s.t. } \boldsymbol{W}_i = \boldsymbol{V}, \boldsymbol{W}_i^\top \boldsymbol{W}_i = \boldsymbol{I}$$

一、ADMM 算法推导

1. 增广拉格朗日函数

引入对偶乘子 $\boldsymbol{\Lambda}_i \in \mathbb{R}^{p \times k}$，增广拉格朗日：

$$\mathcal{L}_\rho(\{\boldsymbol{W}_i,\boldsymbol{S}_i\},\boldsymbol{V},\{\boldsymbol{\Lambda}_i\}) = \sum_{i=1}^{d} \left[\left\| (\boldsymbol{I} - \boldsymbol{W}_i\boldsymbol{W}_i^\top)(\boldsymbol{X}_i - \boldsymbol{S}_i) \right\|_F^2 + \alpha\|\boldsymbol{S}_i\|_1 + \beta\|\boldsymbol{W}_i\|_{2,1} + \langle \boldsymbol{\Lambda}_i, \boldsymbol{W}_i - \boldsymbol{V} \rangle + \frac{\rho}{2}\|\boldsymbol{W}_i - \boldsymbol{V}\|_F^2 \right]$$

ADMM 四块交替迭代：

1. 客户端并行更新 $\boldsymbol{S}_i$
2. 客户端并行求解带 Stiefel 约束的 $\boldsymbol{W}_i$
3. 服务器聚合更新全局 $\boldsymbol{V}$
4. 更新对偶变量 $\boldsymbol{\Lambda}_i$

2. 子问题闭式 / 数值解

(1) $\boldsymbol{S}_i$ 更新（逐元素软阈值）

固定其余变量，令 $\boldsymbol{Y}_i = \boldsymbol{X}_i - \boldsymbol{S}_i$，$\boldsymbol{P}_i = \boldsymbol{I} - \boldsymbol{W}_i\boldsymbol{W}_i^\top$ 幂等投影阵：

$$\min_{\boldsymbol{S}_i} \|\boldsymbol{P}_i(\boldsymbol{X}_i - \boldsymbol{S}_i)\|_F^2 + \alpha\|\boldsymbol{S}_i\|_1$$

闭式解：

$$\boldsymbol{S}_i = \mathcal{S}_\alpha\big(\boldsymbol{X}_i - \boldsymbol{W}_i\boldsymbol{W}_i^\top\boldsymbol{X}_i\big)$$

软阈值算子：$\mathcal{S}_\tau(z) = \operatorname{sign}(z)\max(|z| - \tau, 0)$

python

```python
import numpy as np

# ==================== 算子工具函数 ====================
def soft_threshold(x, tau):
    """L1软阈值"""
    return np.sign(x) * np.maximum(np.abs(x) - tau, 0.0)

def row_21_prox(W, beta):
    """L2,1行近端算子：每行L2范数收缩"""
    p, k = W.shape
    for r in range(p):
        norm_r = np.linalg.norm(W[r, :], ord=2)
        if norm_r > 1e-8:
            scale = max(1 - beta / norm_r, 0.0)
            W[r, :] *= scale
    return W

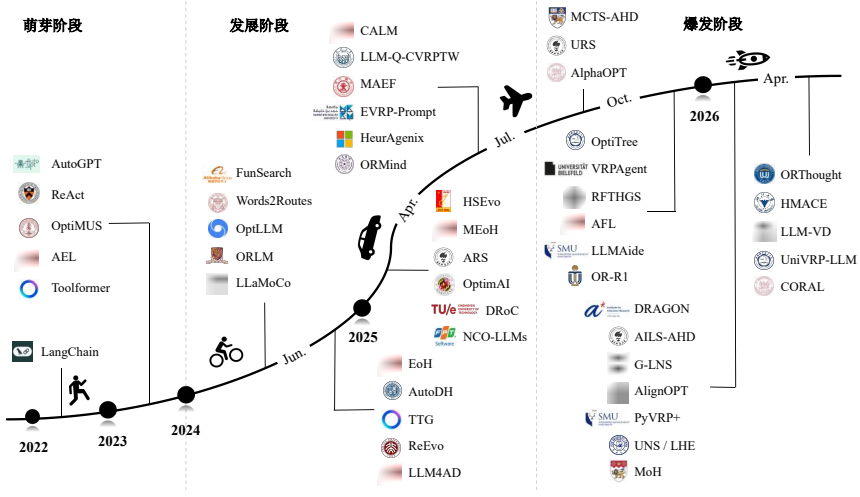
def stiefel_retraction(W):
    """SVD极值型Stiefel流形 W^T W = I"""
    U, S, Vt = np.linalg.svd(W, full_matrices=False)
    return U @ Vt

def stiefel_pgd(C, V_ref, Lambda, rho, beta, lr=1e-2, inner_it
    """
    Stiefel流形近端梯度下降求解W
    min -tr(W^T C) + 0.5*rho||W - (V_ref - Lambda/rho)||_F^2 +
    """
    p, k = C.shape[0], V_ref.shape[1]
    W = np.random.randn(p, k)
    W = stiefel_retraction(W)
    target = V_ref - Lambda / rho

    for _ in range(inner_iter):
        # 欧氏梯度
        grad = -C + rho * (W - target)
        W = W - lr * grad
        # L2,1近端
        W = row_21_prox(W, beta * lr)
        # 流形回缩近交化
        W = stiefel_retraction(W)
    return W
```

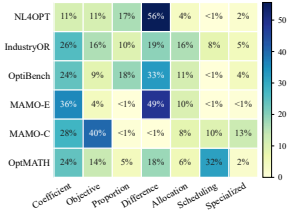
VRP

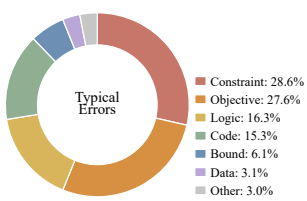
- Xiu-Shen-Zhu et al, A Survey of Large Language Models for Vehicle Routing Problems (in Chinese), CTA, minor revision

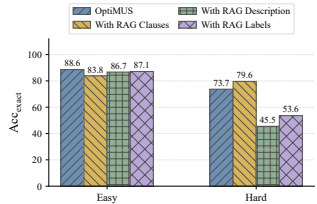


GraphRAG

- Edge-Trinh-Cheng et al, From Local to Global: A GraphRAG Approach to Query-Focused Summarization arXiv:2404.16130
- Gutiérrez-Shu-Gu et al, HippoRAG: Neurobiologically Inspired Long-Term Memory for Large Language Models, NeurIPS, 2024
- Guo-Xia-Yu et al, LightRAG: Simple and Fast Retrieval-Augmented Generation, EMNLP, 2025
- Zhu-Huang-Wang et al, Graph-Based Approaches and Functionalities in Retrieval-Augmented Generation: A Comprehensive Survey, ACM Computing Surveys, 2026







Example

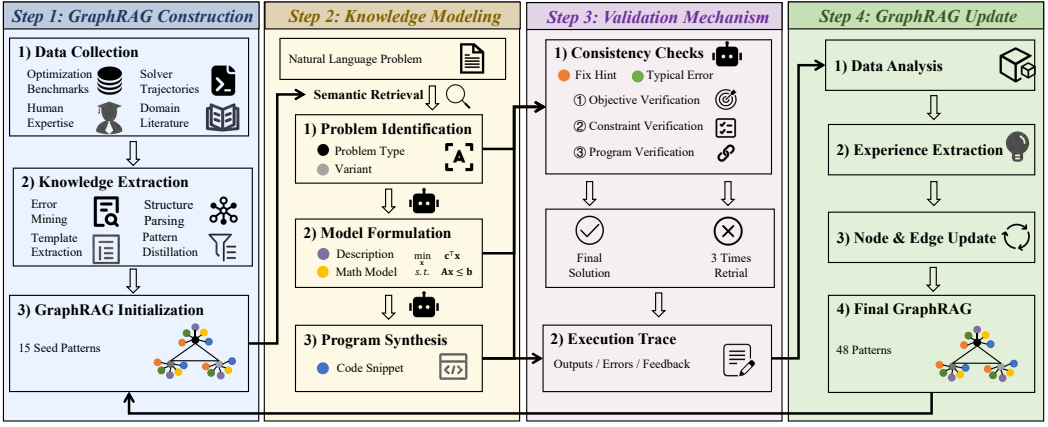
- Problem Type: Vehicle routing problem (VRP)
- Variant: Capacitated vehicle routing problem (CVRP)
- Description: Design vehicle delivery routes to serve all customers with minimal travel cost under vehicle capacity constraints
- Math Model:

$$
\begin{aligned}
\min_{x} \quad & \sum_{k \in K} \sum_{i \in V} \sum_{j \in V} c_{ij} x_{ijk} \\
\text{s.t.} \quad & \sum_{k \in K} \sum_{j \in V} x_{ijk} = 1, && \forall i \in C \\
& \sum_{i \in C} d_i \sum_{j \in V} x_{ijk} \leq Q, && \forall k \in K \\
& x_{ijk} \in \{0, 1\}, && \forall i, j \in V,\ k \in K
\end{aligned}
$$

- Code Snippet:
```
x = addVars(V, V, K, vtype=GRB.BINARY)
min sum(c[i,j] * x[i,j,k] for i in V for j in V for k in K)
addConstrs(each customer visited once), addConstrs(vehicle capacity limits)
```
- Typical Error: Missing customer coverage, ignoring vehicle capacity, disconnected subtours
- Fix Hint: Visit each customer once, add capacity constraints, eliminate subtours

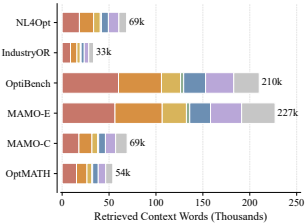
Workflow

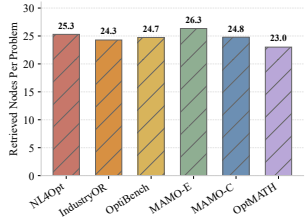
- Xiu-Li-Huang et al, OptGraph: Large Language Models Enhanced Evolutionary Optimization Via Graph Retrieval-Augmented Generation, submitted to ICLR

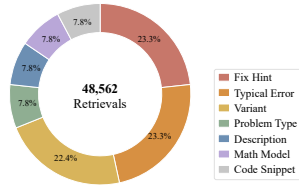


Accuracy

Methods	NL4OPT	IndustryOR	OptiBench	MAMO-E	MAMO-C	OptMATH
GPT-5.1	72.9%/78.0%	43.0%/51.0%	58.0%/65.8%	81.6%/85.8%	47.8%/58.1%	27.7%/30.7%
Gemini-3.1-Pro	73.4%/78.5%	42.0%/50.0%	58.2%/66.0%	82.1%/86.1%	46.8%/57.1%	27.1%/30.1%
DeepSeek-V3.2	72.4%/77.6%	44.0%/50.0%	57.4%/65.1%	81.0%/85.0%	48.3%/57.6%	26.5%/30.1%
Qwen3.6-Plus	69.6%/74.8%	42.0%/49.0%	55.4%/62.6%	78.0%/81.8%	46.3%/55.7%	27.1%/28.9%
CoE	77.1%/83.6%	51.0%/61.0%	65.5%/73.2%	93.5%/93.8%	67.0%/70.3%	40.4%/43.4%
OptiMUS	86.9%/93.9%	54.0%/60.0%	63.0%/72.1%	87.8%/89.0%	52.2%/70.4%	38.5%/41.3%
OptiTree	84.1%/92.5%	64.0%/71.0%	62.5%/71.9%	90.7%/91.2%	65.4%/72.8%	31.3%/33.7%
Lean-LLM-OPT	86.5%/94.9%	55.0%/63.0%	61.8%/70.3%	93.8%/94.2%	57.6%/70.0%	44.5%/45.2%
OptGraph (Ours)	**86.9%/95.3%**	**71.0%/81.0%**	**66.0%/78.2%**	**93.9%/94.4%**	**76.4%/89.2%**	**58.4%/60.8%**

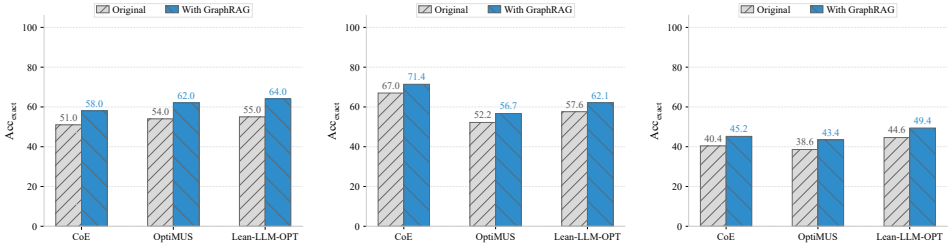






Remarks

- Plug-and-play effects on IndustryOR, MAMO-C, and OptMATH

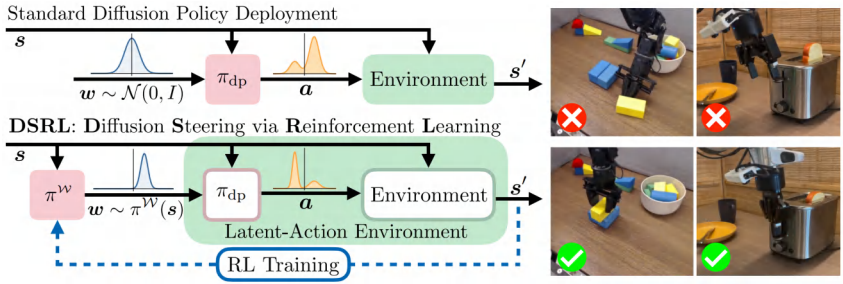


- Computational costs in average

Methods	Tokens (k)	Time (s)	$\mathrm{Acc}_{\mathrm{exact}}$	$\mathrm{Acc}_{5\%}$
CoE	**1.3**	79.1	65.8%	70.9%
OptiMUS	15.9	111.1	63.7%	71.1%
OptiTree	4.6	**14.3**	66.3%	72.2%
Lean-LLM-OPT	1.5	17.1	66.5%	72.9%
OptGraph (Ours)	9.7	93.9	**75.4%**	83.2%

Part VI

- Wagenmaker-Nakamoto-Zhang et al, Steering Your Diffusion Policy with Latent Space Reinforcement Learning, CoRL, 2025



- How to improve the action representation?
- How to develop a lightweight residual modulation mechanism?

Methodology

- Zhang-Sun-Xiu, Beyond Noise Steering: Dual-Latent Space Reinforcement Learning for Generative Robot Policy, submitted to ICRA

$$(z_t, u_t) \sim \pi_\theta(\cdot \mid o_t), \;\; f_t = \mathcal{A}_\omega(u_t)$$

$$\Downarrow$$

$$a_t = G_\phi(o_t, z_t; f_t)$$

- How to develop a lightweight residual modulation mechanism?

$$H_{t,\kappa}^{(l)} = \mathcal{T}_\phi^{(l)}(\widetilde{H}_{t,\kappa}^{(l-1)}; e_t, \kappa), \;\; \delta_t = \lambda_{\text{inj}}\, \mathcal{B}(f_t)$$

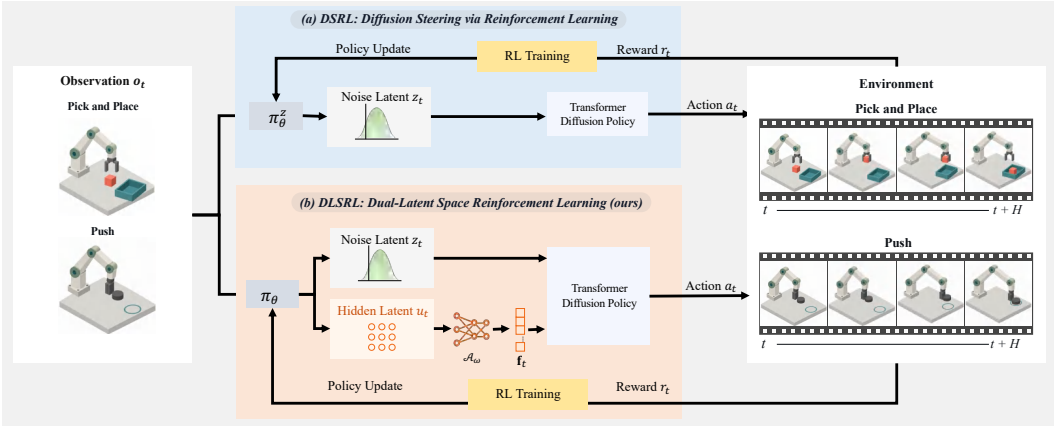
$$\Downarrow$$

$$\widetilde{H}_{t,\kappa}^{(l)} = H_{t,\kappa}^{(l)} + \delta_t$$

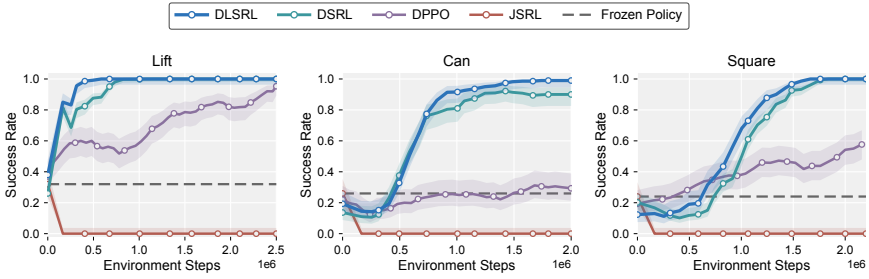
- Latent-space optimization

$$\mathcal{L}_{\text{actor}}(\theta, \omega) = E_{o\sim\mathcal{D},(z,u)\sim\pi_\theta(\cdot|o)}\Big[\alpha \log \pi_\theta^z(z \mid o) - Q_\nu^{\text{lat}}(o, \xi)\Big] + \mathcal{R}_{\text{aux}}$$

Architecture

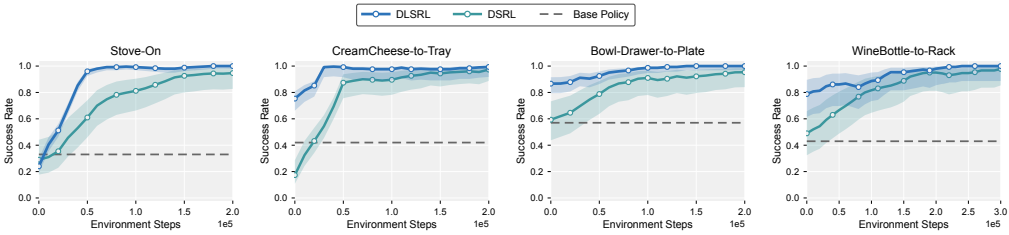


RoboMimic


DLSRL
DSRL
DPPO
JSRL
Frozen Policy
Lift
Can
Square
Success Rate
Environment Steps
1e6



LIBERO

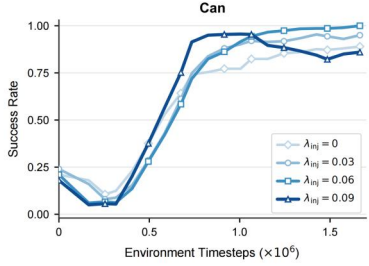


Tasks	DLSRL ↓	DSRL ↓	Reduction
Stove-On	**65.90 ± 1.12**	88.34 ± 16.42	25.4%
CreamCheese-to-Tray	**157.20 ± 5.77**	203.44 ± 12.58	22.7%
Bowl-Drawer-to-Plate	**97.06 ± 0.22**	125.80 ± 29.18	22.8%
WineBottle-to-Rack	**95.96 ± 0.65**	113.26 ± 8.97	15.3%
Macro Average	**104.03**	132.71	21.6%

Remarks

- How to develop fast and robust policy optimization?
- How to choose λ_{inj} by learning approaches?





- More experiments on real-world robots!

Outline

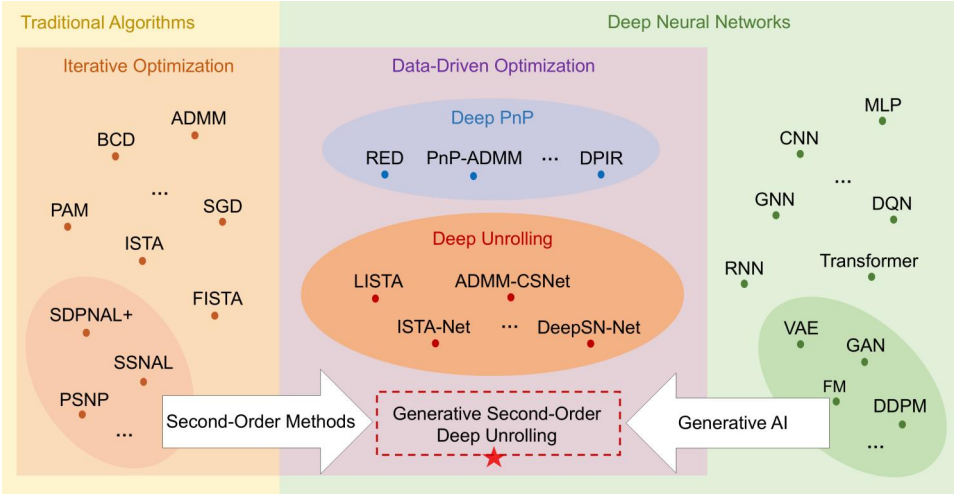
Introduction

Recent Advances

Future Work

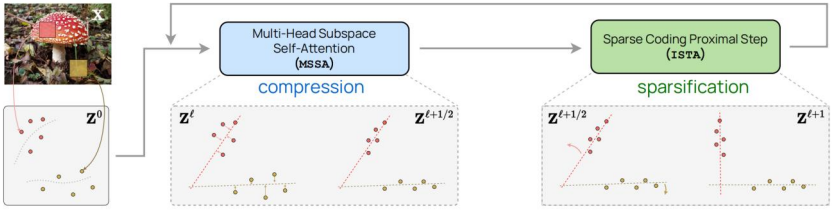
Future Work

- Deep Unrolling + Second-order Optimization + Generative AI



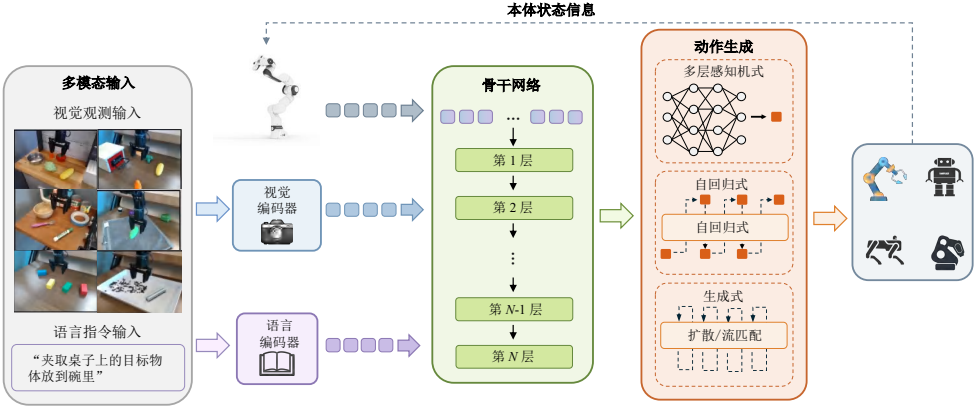
Future Work

- How to explain Transformer?
  - Yang-Huang-Wipf, Transformers from An Optimization Perspective, NeurIPS, 2022
  - Yu-Buchanan-Pai et al, White-box Transformers via Sparse Rate Reduction, NeurIPS, 2023
  - Wang-Lu-Yu et al, Attention-Only Transformers via Unrolled Subspace Denoising, ICML, 2025
  - Porras-Hadou-Ribeiro, A Constrained Optimization Perspective of Unrolled Transformers, ICML, 2026
  - Tai-Liu-Li et al, A Mathematical Explanation of Transformers, SIIMS, 2026



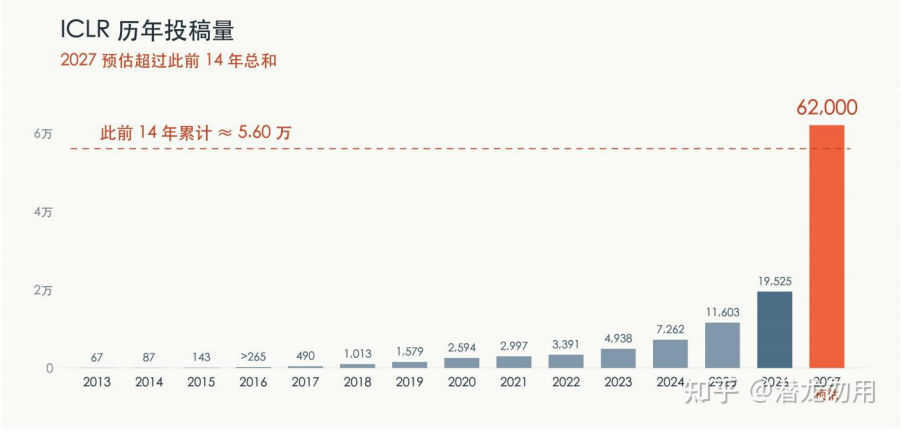
Future Work

- Xiu-Xu-Zhu et al, Review of Compression Methods for Vision-Language-Action Models (in Chinese), submitted to Acta Automatica Sinica



Future Work

- Xiu, Artificial Intelligence-Driven Optimization: From Algorithms to Applications (in Chinese), working in process



https://xianchaoxiu.github.io